

AWS Certified Generative AI Developer — Professional

Domain 4, Task 4.2 — Application performance optimisation · Revision notes

4.1 vs 4.2

Task	Optimises for
4.1	Cost — spend less money
4.2	Speed — lower latency, higher throughput

These often pull in opposite directions: caching helps both, but provisioned throughput buys speed with money.

Genuinely new in this task: HNSW vs IVF and their tuning parameters, latency-optimized Bedrock models, SageMaker Inference Recommender, CloudWatch Evidently, EC2 Inferentia, predictive scaling, custom scoring functions.

1. Responsive AI Systems

Actual latency is how long it really took. **Perceived latency** is how long it felt. Both can be improved — and they are improved by different techniques.

Pre-computation

```
identify predictable queries
-> process in advance with Lambda
-> store in DynamoDB
-> request arrives -> straight lookup
```

A model call takes ~3 seconds; a DynamoDB lookup takes ~10 milliseconds. Suits FAQs, product descriptions, daily reports and common calculations. Configure appropriate **read capacity units** to avoid throttling.

Store	Latency	Use for
DynamoDB	Single-digit ms	Persistent, larger datasets
ElastiCache (Redis OSS)	Sub-millisecond	Hot, frequently accessed responses

Latency-optimized Bedrock models

Bedrock offers latency-optimized versions of certain models — faster inference, with an explicit **trade-off between response quality and speed**. Suited to time-sensitive applications where a small quality concession is acceptable (the 4-5 second clinical queries in Task 2.4 are exactly this situation).

Response streaming — the perceived-latency lever

```
without streaming: 8-second spinner, then the full answer
with streaming:    first words in 0.5 seconds
```

Total time is unchanged at 8 seconds. Actual latency did not improve at all — but the experience is transformed. Streaming is the cheapest perceived-performance win available.

Parallel requests — Step Functions

```
sequential: 3 calls x 2s = 6 seconds
parallel:   3 calls at once = 2 seconds
```

Only works when the operations are independent — if B needs A's result, they cannot run in parallel.

Benchmarking and A/B testing

- Systematic benchmarking frameworks in **CloudWatch** compare model performance across configurations
- **CloudWatch Evidently** — purpose-built for feature flags and A/B experiments with built-in statistical analysis, used here to evaluate latency improvements in production. (Task 2.4 used AppConfig for rollout; Evidently is the experimentation service.)

2. Retrieval Performance

The most technical section — an extension of Task 1.4.

Vector index algorithms

Algorithm	How it works	Trade-off
HNSW Hierarchical Navigable Small World	Graph-based, organised in layers — coarse connections above, fine below. Search descends the layers to the nearest neighbours.	Very fast with high recall, but memory-hungry and slower to build
IVF Inverted File Index	Cluster-based — find the nearest clusters, search only within them	Lower memory , better for very large datasets; slightly less accurate

need speed + recall, memory available -> HNSW

very large dataset, memory constrained -> IVF

HNSW parameters

Parameter	Effect of increasing it
m	Connections per node — better recall, more memory , slower index build
ef_construction	Candidates considered while building — better index quality, slower build (no effect on search speed)
ef_search	Candidates considered at query time — better recall, slower search

m, ef_construction -> build time and memory

ef_search -> query time and recall

Hierarchical indices and query preprocessing

- **Hierarchical indices:** top-level index for general information, lower-level indices for detail — the coarse-to-fine pattern from Task 1.4
- **Query preprocessing in Lambda** before the vector search: normalise, expand (via Bedrock, Task 1.5), decompose, correct spelling. A better query means better results and fewer retries.

Hybrid search and custom scoring

final score = relevance x w1

+ recency x w2

+ authority x w3

Hybrid search (semantic + keyword) comes from Task 1.5. **Custom scoring functions** are the new element: similarity alone is insufficient, because a perfectly relevant 2010 document may simply be **out of date**. This is Task 1.4's precedential-value reranking, implemented at index level.

Frequent queries can also be cached in **ElastiCache** to cut retrieval latency.

3. Throughput Optimisation

Measures	
Latency	How fast <i>one</i> request completes
Throughput	How many requests per second

They often oppose each other — batching raises throughput while raising per-request latency.

Token processing and batching

- Efficient tokenization and batching, informed by Bedrock's token counting. In 4.1 fewer tokens meant less cost; **here fewer tokens mean faster processing and more requests served**
- Batch inference** for non-interactive workloads; **SQS** to queue and batch during peak load

Concurrent invocation control

Unlimited parallelism causes throttling — 429s, retries, and a net slowdown. Step Functions Map state with `MaxConcurrency` caps how many run at once; the rest wait.

```
MaxConcurrency: 10
-> only 10 execute concurrently
-> plus rate limiting and backoff (Task 2.4)
```

Request prioritisation

Critical requests must be served first under load — separate priority queues, or SQS message attributes. Same pattern as the emergency priority queuing in Task 2.4.

4. FM Parameter Tuning

Parameter	Controls
temperature	Randomness — 0 deterministic, 1 creative
top_k	Choose from the top K tokens — a fixed count
top_p	Nucleus sampling — enough tokens to reach probability mass p

```
top_k = 50 -> always 50 candidates, however likely they are
top_p = 0.9 -> however many tokens make up 90% of probability
```

top_p is adaptive: when the model is confident it considers few candidates; when uncertain, more. That is why it is often the better control. And as in Task 1.3 — **tune temperature or top_p, not both.**

Tuning workflow: test configurations systematically → compare with **Model Evaluation** → A/B test with **Evidently** → adopt the optimum. Specialised prompt templates per use case may each warrant their own parameters.

5. Efficient Resource Allocation

Use case: financial market infrastructure providers running production GenAI.

```
Why GenAI workloads are different:
significant compute and memory requirements
variable latency under high inference demand
operate under specific throughput QUOTAS
don't follow conventional infrastructure patterns
-> traditional FinOps approaches are insufficient
```

SageMaker Inference Recommender

```
automated load testing across various loads
-> recommends the OPTIMAL INSTANCE TYPE
-> balancing cost-effectiveness and performance
-> covers both real-time and serverless options
```

This is the concrete tool behind 4.1's "capacity planning" — **measure rather than guess.**

Provisioned Throughput

- Dedicated infrastructure with **higher and more stable throughput than default on-demand quotas**
- Cross-region inference profiles** distribute demand across regions

Utilisation monitoring — GenAI specific

```
track resource metrics
monitor TOKEN LENGTHS of prompts and responses
identify IDLE periods -> scale down or suspend endpoints
SageMaker Model Monitor -> performance and data quality
```

Monitoring token lengths is specific to GenAI. In a conventional application request size is roughly fixed; here it is the primary driver of utilisation.

Auto scaling for GenAI traffic

```
SageMaker endpoints -> EC2 Auto Scaling groups behind load balancers
predictable demand -> deploy larger instances PROACTIVELY
queuing between app and model -> prevents request denial
                                under throughput constraints
event-driven messaging for high-demand architectures
```

Predictive scaling uses historical usage to anticipate spikes. The transcript's example is a trading platform scaling for **market open and close** (scheduled) as well as unexpected surges. **Reactive scaling arrives too late** — by the time capacity is added, the spike has passed.

Hardware and automation

- **EC2 Inferentia** — purpose-built inference silicon, cheaper and more efficient than general GPUs for inference (Trainium is the training counterpart). Large models are distributed across multiple instances.
- **Infrastructure automation:** AI agents analyse application requirements and generate configurations, delivered as IaC (e.g. Terraform) following established patterns

6. System Performance Optimisation

- **X-Ray** — profile API calls for **prompt-completion patterns**, building on the subsegments and annotations from Task 2.4; the focus here is which prompt shapes are slow
- **CloudWatch Insights** — analyse API call patterns for optimisation opportunities
- **Vector query optimisation** and query caching (sections 2 and 4.1)
- **CloudFront** for edge caching to global users; **API Gateway** for efficient request routing
- Real-time performance dashboards in CloudWatch

The optimisation loop

```
Capacity Planning
  (Inference Recommender + Provisioned Throughput)
  | informs
Monitoring
  (CloudWatch metrics + SageMaker Model Monitor)
  | triggers
GenAI-Optimized Autoscaling
  (event-driven scaling + request queuing)
  | enhances
Resource Optimization
  (service quota management + EC2 Inferentia)
  |
  +--- FEEDBACK LOOP back to Capacity Planning
```

Service limits and feature availability change. Verify specific numbers against current AWS documentation before the exam.